Neural CPR policies in OPM Flow

Project summary

Status 2026-10-20, with an update from 2026-10-26

This project tests whether small neural networks, making some of the linear solver's decisions while the simulation runs, can make the OPM Flow reservoir simulator faster. The physics is unchanged; only how fast Flow gets to the answer changes.

In short: two of the three policies work. The *neural AMG policy* (picking pressure-AMG settings) and the *neural abort policy* (giving up early on timesteps that will fail anyway) are each worth roughly 5–25% on most decks, and are usually best used together. The third, the *CPR variant policy*, is neutral or worse and is not recommended. The gains carry over to decks the models never saw during training, with two exceptions where they are neutral or slightly negative.

1 The problem

Flow advances a reservoir simulation in timesteps. Each timestep needs several Newton iterations, and each Newton iteration solves a large linear system with the CPR preconditioner. Two things cost a lot of time:

1. **Solver settings.** CPR has many settings, such as the variant and how the pressure AMG part is built. Flow always uses the same defaults, even when another choice would be faster for the current state of the simulation.
2. **Failed timesteps.** When a timestep does not converge, Flow keeps trying for up to 20 Newton iterations, gives up, halves the step ("chops") and tries again. All the work on the failed attempt is wasted. On SPE11 about 15% of the runtime goes to chopped attempts.

2 What was built

Three neural policies were added inside Flow (`opm-simulators`, branch `neural-cpr-control`). Each is a small network that decides exactly one thing, at one specific point in the solve. They are independent and can be enabled separately or together.

	Neural CPR variant policy	**Neural AMG policy**	**Neural abort policy**
Attacks	solver settings	solver settings	wasted work
Runs	before each solve	at each CPR setup rebuild	before each solve
Chooses between	24 CPR configurations	9 pressure-AMG settings	continue / give up
Network	classifier over 24 labels	ensemble of 5, 12 → 32 → 32 → 9	14 → 16 → 1, sigmoid
Verdict	**not recommended**	**recommended**	**recommended where timesteps fail**

2.1 Neural CPR variant policy ("neural-v3", the earlier approach)

A classifier that picks the whole CPR configuration for each solve, from 24 labels spanning four axes: the pressure-decoupling weight type (quasi-IMPES / true-IMPES / analytic true-IMPES), whether to use CPRW well weights, the fine-level smoother, and the coarse AMG smoother. A softmax confidence gate keeps it on Flow's default when the network is unsure.

This was the original approach and it did not pay off — see the findings below. Flow's default is already the best or tied-best of the 24 on every deck tested, so there is little room to win and real room to lose.

2.2 Neural AMG policy

Rather than choosing the whole configuration, this one tunes just two numbers inside the pressure AMG: the prolongation damping (1.0 / 1.3 / 1.6) and the coarsening target (1200 / 600 / 300), nine combinations in all. It runs only when the CPR setup is being rebuilt, not on every solve, so its overhead is small.

It is a *cost model*, not a classifier: the network predicts the log-cost of each of the nine settings, and the policy leaves Flow's default only when some setting is predicted to beat it by more than 5%. Being anchored on the default this way is what keeps it safe on decks it has not seen — ties and near-ties stay on the default.

2.3 Neural abort policy

This one does not change how a solve is done; it decides whether to bother. Before each linear solve it estimates the probability that the current timestep attempt is going to end in a chop regardless. Above 95% it sets the iteration budget to zero, so the solve fails immediately and Flow chops straight away instead of first grinding through the remaining Newton iterations and a full failing solve.

Two safety guards sit outside the model, because aborting itself causes a chop and chops make the next attempt look doomed — an unguarded policy can chop itself into Flow's cut limit, which was observed on an unseen deck. So it never aborts before an attempt's first solve (no evidence from the attempt yet), and never once three attempts in a row have been chopped (Flow's own logic takes over).

Note that this policy can only help on decks that actually waste work. Where no timestep ever fails, it never fires and its per-solve check is pure overhead — about 5% of the linear solve time.

2.4 Rules shared by all three

- Flow's default settings are always the fallback: if no model loads, Flow behaves as upstream.
- Solver options that don't apply to a deck are blocked — for example, analytic true-IMPES is forbidden on decks with no oil phase, and the CPRW-versus-CPR choice is forced to match whether the deck has wells.
- Under MPI, rank 0 decides and broadcasts the choice. Each rank sees only its own matrix partition and could otherwise legitimately disagree, leaving ranks to build mismatched solver objects and deadlock.

3 How the models were trained

1. **Sweep.** All solver configurations were run on Norne, SPE9 and SPE11A, logging features and timing at every Newton step.
2. **Label.** For each solve, the fastest configuration was marked as the right answer. The abort model was labelled from which attempts were chopped.
3. **Train.** Small networks were trained on those labels (`opm-simulators/doc/neural_cpr/`).

SPE11B, SPE10, Drogon and Sleipner were kept out of training and used only to test how the models carry over to new decks.

4 How it was tested

- `compare_neural_cpr.sh`: plain Flow and each neural policy on one deck, one after another (see `COMPARE_NEURAL_CPR_README.md`).
- `bench_neural_cpr.sh`: the whole pipeline (sweep, label, train, benchmark) over many decks; with `FRESH_SWEEP=0 SKIP_TRAIN=1` it only benchmarks.

All benchmark runs below use 32 MPI ranks × 2 threads with `-bind-to none` on `cno-0008`.

5 What was found

Change in simulation time compared with plain Flow (negative = faster).

Deck	Used in training?	Neural AMG	Neural abort	Both	Source
SPE11A	yes	−25.5%	−10.0%	−32.2%	3× repeats, Oct 16
SPE9	yes	−10.6%	−2.2%	−10.7%	3× repeats, Oct 16
Norne	yes	−8.0%	+2.4%	−0.8%	3× repeats, Oct 16
Norne MSW (1A)	variant of Norne	−21.0%	−12.1%	−23.2%	single run, Oct 20
SPE10 MOD01	no	−19.9%	−28.5%	−28.1%	single run, Oct 20
SPE11B (thermal)	no	−4.1%	−7.0%	−10.1%	3× repeats, Oct 16
Drogon HIST	no	+1.6%	−2.0%	−2.6%	single run, Oct 20 (15 s run)
Sleipner	no	+4.0% to +5.5%	−2.2% to −3.7%	+1.7% to +4.0%	2 runs, Oct 18

- **Neural AMG policy:** clear gains on 6 of the 8 decks. Slower on Sleipner, where its AMG choices nearly double the linear iterations, and about neutral on Drogon.
- **Neural abort policy:** helps most where many timesteps fail (SPE10, Norne MSW, SPE11). On Norne it never fires, but its per-solve check still costs about 5% of the linear solve time.

- **Both together:** the best or close to the best on most decks. The exceptions are Norne, where the abort check's overhead cancels the AMG gain, and Sleipner, where adding the abort policy causes extra chops.
- **CPR variant policy (`neural-v3`):** neutral or worse — +1.6% on Norne, +4% on SPE9, −3% on SPE10 MOD01 and +48% on Norne MSW. This is the policy with nothing to win: Flow's default is already the best or tied-best of the 24 configurations on every deck tested.
- **Single runs vs repeats:** the Oct 20 single runs on Norne and SPE9 agree with the 3× repeats for the AMG policy (−14% and −23%). The abort policy came out slightly better (−4% on both), within run-to-run noise. On Sleipner, repeated runs differed by up to 3%.

Not yet measured:

- **Drogon PRED** failed in every arm. It restarts from the HIST run, and the HIST restart file is graphics-only or missing the well arrays it needs. HIST must be run with standard restart output.
- **SPE10 model1:** the deck file `decks/spe10model1/SPE10_MODEL1.DATA` is missing.
- **SPE11A and SPE11B in the Oct 20 bench** were still running when this was written.

6 Update 2026-10-26: two runs on cno-0009

Two `compare_neural_cpr.sh` runs using the already-trained models, with no retraining. They differ from everything above in two ways that matter when reading the numbers: they ran on a different machine (`cno-0009` rather than `cno-0008`), and there is **only one run per arm**, so nothing here averages out run-to-run noise. Both runs used 32 MPI ranks, with the four arms run one after another.

6.1 SPE11B — `tmp/compare_neural_cpr/SPE11B_20261026_163308`

101040 active cells, ∼3 min per arm. All arms exit 0 and reach the same final report step (16/17, day 390550), so the comparison is like-for-like.

Arm	Simulation time	Δ vs default	Linear solve	Linear iterations	Wasted lin. iterations	Abort fires
default	184.96 s	—	109.15 s	23577	5248 (22.3%)	—
neural-abort	180.44 s	−2.4%	103.87 s	21604	2770 (12.8%)	37
neural-amg	176.49 s	−4.6%	100.26 s	21345	4311 (20.2%)	—
neural-amg+abort	**170.81 s**	**−7.6%**	95.48 s	20002	3496 (17.5%)	45

The two policies save time in different ways, which is why their gains add up rather than overlap: neural-amg cuts the *total* number of linear iterations (23577 → 21345) by preconditioning better, while neural-abort cuts the *wasted* share of them (22.3% → 12.8%) by not finishing attempts that are going to be thrown away. Used together they reach the lowest iteration count of any arm.

The combined result is close to the Oct 16 3× repeats for this deck (−10.1%); the abort arm on its own came out weaker here (−2.4% vs −7.0%), which single runs cannot separate from noise.

6.2 SPE9 — `tmp/compare_neural_cpr/SPE9_20261026_164620`

9000 active cells, ∼3 s per arm. **No timing conclusion should be drawn from this run.**

Arm	Simulation time	Δ vs default	Linear solve	Linear iterations	Wasted lin. iterations
default	2.97 s	—	1.53 s	432	0 (0.0%)
neural-abort	2.98 s	+0.3%	1.52 s	432	0 (0.0%)
neural-amg	2.83 s	−4.7%	1.18 s	417	0 (0.0%)
neural-amg+abort	2.49 s	−16.2%	1.09 s	417	0 (0.0%)

The reason is that this deck never has a convergence failure: wasted iterations are 0.0% in every arm, and the abort policy fired 0 times even where it was enabled. There is simply nothing for it to abort. That leaves two pairs of arms doing identical work:

- `neural-abort` does the same work as `default` — both 432 linear iterations, 234 Newton iterations, 327 linearizations.
- `neural-amg+abort` does the same work as `neural-amg` — both 417 / 237 / 330.

Yet the second pair's timings differ by 11 percentage points (−4.7% against −16.2%) for what is the same computation. That gap is measurement noise, not a policy effect: at 3 s wall on 32 ranks, process startup and MPI variance dominate the timing.

The one real signal is that neural-amg changes the solve: 432 → 417 linear iterations, consistently in both arms that enable it. The deck is too short to turn that into a trustworthy wall-clock number.

Takeaway for the test matrix: a deck with no wasted iterations cannot show an abort-policy effect, and a deck this short cannot resolve an AMG effect. Such decks are only worth running with many repeats, or not at all.

7 Where things are

What	Where
Code	`opm-simulators`, local branch `neural-cpr-control` (tag `neural-cpr-snapshot-20261017`)
Models	`tmp/amg_policy/cpr_amg_policy.model`, `tmp/abort_policy/abort_spe11a_norne_spe9.model`
Full snapshot, restore steps, detailed notes	`neural_cpr_snapshot_20261015/NOTES.md`
Per-deck comparison runs	`tmp/compare_neural_cpr/` (e.g. `SPE11B_20261026_163308/`)
Bench runs	`tmp/bench_neural_cpr/`